

Defect Detection in Fabrics using MATLAB

Akhil Mathew Cherian

INDEX

1. Introduction
2. Problem Definition
3. Solution
4. Dataset
5. Coding
6. Examples
7. Challenges
8. Conclusion

Introduction

In the textile industry, identifying fabric faults is a key duty because it ensures the final product's quality. The automatic detection of fabric defects using computer vision and image processing techniques has become more and more common in recent years.

Given that it provides a comprehensive collection of image-processing tools and operations, MATLAB is the ideal software for developing algorithms for identifying fabric flaws. The fundamental idea is to train a machine learning model to automatically detect faults using attributes that may be retrieved from the fabric pictures.

Problem Definition

Any phase of the fabric production process, such as the creation of the yarn, weaving, and finishing, is susceptible to errors. These faults are categorised as major or minor depending on how serious they are and how they affect the outcome. Finding and identifying these flaws is a difficult undertaking that involves careful inspection and examination of the fabric's surface. Traditional inspection techniques include laborious, time-consuming, and error-prone visual inspections performed by humans. A manual check might also miss some flaws that are invisible to the unaided eye. As a result, the textile industry's quality control process will be greatly improved by applying computer vision techniques to develop an automated system for classifying and recognising fabric faults. The following problems can arise if not using an automated systems.

Inconsistent quality control: If there is no automatic technique in place to discover defects, the quality of the final output could differ. Human inspectors may miss problems or have various definitions of what constitutes a flaw, which can result in variations in the final product's quality.

Increased costs: The manual inspection of fabric samples is a labor-intensive, resource-intensive process that takes a lot of time. Businesses could have to pay additional employees to inspect their products in the absence of an automated system, which would increase costs.

Reduced efficiency: The laborious manual inspection of fabric samples might slow down the production process. Automated systems are more effective since they are quicker and more accurate at finding problems.

Inaccurate defect detection: Human inspectors may miss minor or undetectable faults in the fabric, leading to inaccurate fault detection. Customers could obtain inferior goods as a result, harming the reputation of the business and bringing about losses.

Solution

This model will detect the stains and holes in a fabric image using image processing techniques in MATLAB. The holes or stains represent a fabric fault that needs to be found and isolated in order to ensure the quality of the end product.

The main objective is to develop a machine that can identify and point out the holes or stains' location in a fabric image so that it may be analyzed and evaluated in more detail. Th uses various image processing methods, including filtering, thresholding, and object removal, to separate the burn hole region from the input fabric image.

This model's method can be used to mend a variety of fabric photos with stains or holes, regardless of how big, where, or how the burn hole is. By automating the process of problem identification, the model can improve the efficacy and precision of quality control procedures in the textile industry. The model is useful in many scenarios.

Automated quality control: In the textile industry, quality control is essential to ensuring that the final product conforms with the required specifications. By implementing automatic defect detection methods like the one the model provides, businesses can save time and money while maintaining the quality of their products.

Improved accuracy: Human inspectors might not be able to detect minute or subtle problems, whereas automated methods can detect even the smallest imperfections in the fabric. As a result, the defect diagnosis process is more precise, leading to higher-quality products.

Cost-effective: Manual examination is a part of traditional inspection methods, which can be labour- and time-intensive. Automated approaches, like the one the model offers, can lessen the need for manual inspection and can rapidly and precisely find mistakes.

Research purposes: This can also be used by researchers studying fabric faults to classify and examine various problems, possibly leading to the development of novel strategies to prevent them.

Dataset

The photos in the dataset were taken from Google and include no-defect fabrics as well as ones that have been burned, ripped, and holed. To enable MATLAB processing, we carefully select those photos. The many image types utilised in the dataset are listed below.

Holes





Stains



Burns



Inconsistencies



Normal



Coding

MATLAB is used for the coding of the image detection model.

Loading the image

```
untitled * x +
1      clc
2      clear all
3      close all
4      warning off
5      % Load the fabric image
6      fabric = imread('pf1.jpg');
7      figure;
8      imshow(fabric);
```

This MATLAB programme uses the `imread()` function to load the picture file. It is expected that the picture file is located in MATLAB's current working directory.

Once the image has been loaded, it uses the `imshow()` function to display it in a new figure window. The loaded image is displayed by `Imshow()` in the figure window that was generated by the `figure()` method.

Convert to greyscale

```
9
10
11      % Convert to grayscale
12      gray_fabric = rgb2gray(fabric);
13      figure;
14
15
```

The above MATLAB code turns the previously loaded colour image file into grayscale using the `rgb2gray()` function. The resulting grayscale image is stored in a new variable called `gray_fabric`.

The code displays the grayscale image in a new figure window using the `figure()` and `imshow()` functions. The figure window, which is produced by the `figure()` function, displays the grayscale image when `Imshow()` is called.

Apply Median Filter

```
% Apply a median filter to remove noise
filtered_fabric = medfilt2(gray_fabric, [5 5]);
figure;
```

This MATLAB programme applies a 2D median filter to the grayscale image `gray_fabric` using the `medfilt2()` function. When using the `medfilt2()` function, each pixel in the image is replaced by the median value of its neighbours within a given window size. Since the window size is set to `[5 5]`, the filtering in this case takes place within a 5x5 square window.

The filtered image is stored in a brand-new variable named `filtered_fabric`.

The image is filtered in the code before being displayed in a new figure window using the `figure()` and `imshow()` functions. The filtered image is shown by `imshow()` in the figure window that was made by the `figure()` method.

Threshold the image

```
% Threshold the image using Otsu's method
level = graythresh(filtered_fabric);
bw_fabric = imbinarize(filtered_fabric, level);
figure;
```

This MATLAB application applies a thresholding operation to the previously filtered grayscale image `filtered_fabric` using Otsu's method. The threshold level is calculated using the `graythresh()` function, which bases its results on the histogram of the input image. The selected threshold value is saved in the variable `level`.

The binary image `bw_fabric` is created by applying the threshold to the filtered grayscale image using the `imbinarize()` function after the threshold value has been determined. Which pixels are assigned to 0 (black) and which ones to 1 (white) are determined by the intensity threshold.

To display the binary image in a new figure window, the code employs the `figure()` and `imshow()` functions. In the figure window that was produced by the `figure()` method, `imshow()` displays the binary picture.

Removing Small objects

```
36  
37 % Remove small objects (less than 1000 pixels) from the image  
38 cleaned_fabric = bwareaopen(bw_fabric, 1000);  
39 figure;
```

The binary image `bw_fabric` that has previously been thresholded is removed from the programme using the `bwareaopen()` function in MATLAB. The `bwareaopen()` function removes all connected components (objects) in the binary image that are less than a specified number of pixels in size. The variable `cleaned_fabric` stores the binary picture that is produced after deleting objects with fewer than 1000 pixels.

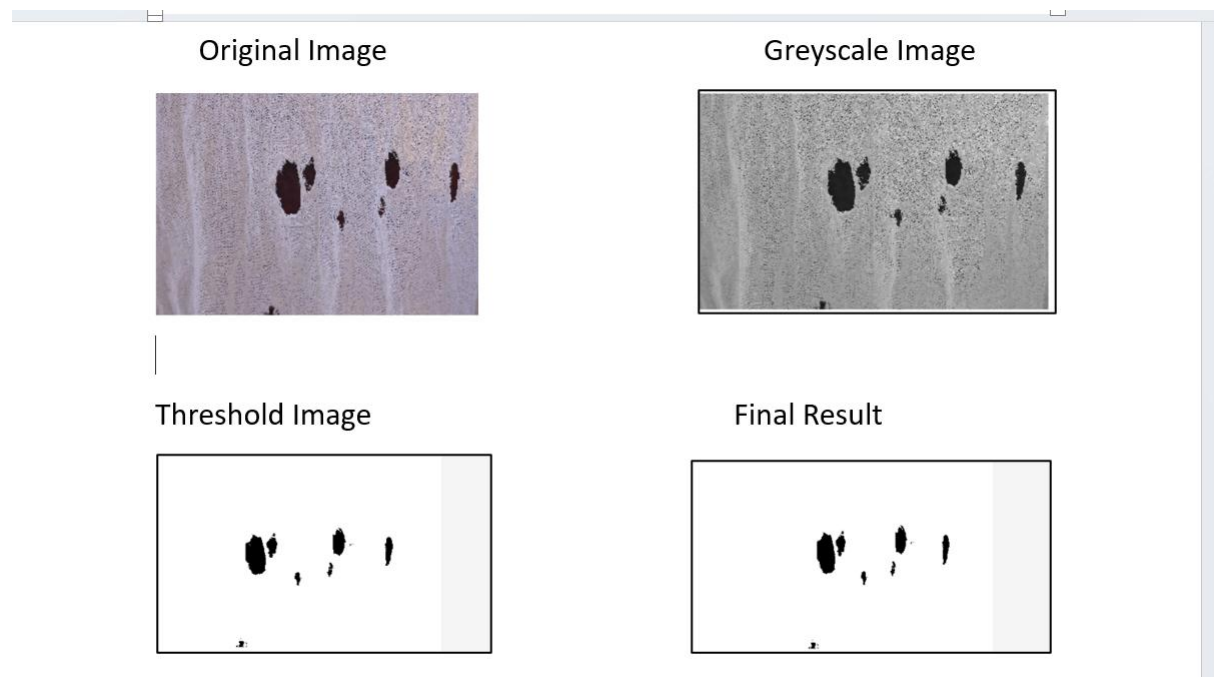
After removing small objects from the binary image, the code uses the `figure()` and `imshow()` functions to display the cleaned binary image in a new figure window. In the freshly constructed figure window produced by `figure()`, the `imshow()` function displays the cleaned binary image.

Plotting the images

```
2  
3  
4 %plotting each image|  
5 imshow(gray_fabric);  
6 imshow(filtered_fabric);  
7 imshow(bw_fabric);  
8 imshow(cleaned_fabric);
```

The above MATLAB code plots each image that we have processed.

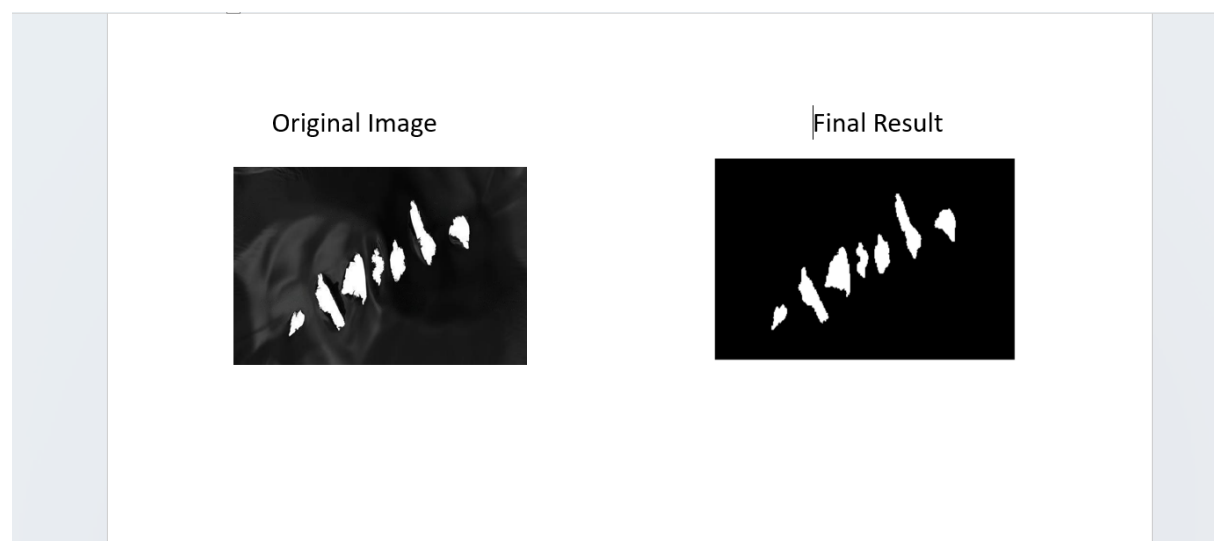
Example of an image in each category



From the above result, we can say that the model is very efficient in finding even little holes in the fabric.

Images and their results

Images with holes



Original Image



Final Result



Original Image



Final Result



Original Image



Final Result

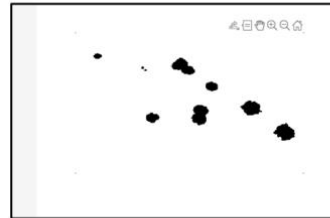


Images with Stains

Original Image



Final Result



Original Image



Final Result



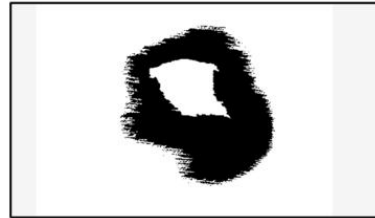
The system can find the stains in a fabric which is very small.

Burnt holes

Original Image



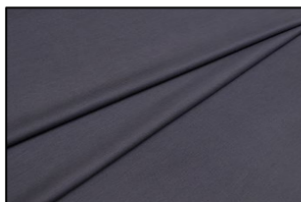
Final Result



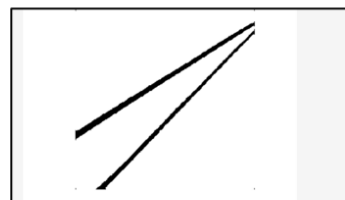
This system can also find burnt holes in a fabric.

Fabrics with Inconsistencies

Original Image



Final Result



This is an example of a fabric with irregular textural features. Texture variations from the desired or anticipated pattern are referred to as textural inconsistencies.

In other words, texture inconsistencies happen when a material's texture changes abruptly or has faults. For instance, "textural inconsistencies" in the context of image analysis can refer to differences in a picture's texture that could influence how it appears or make it challenging to extract crucial information.

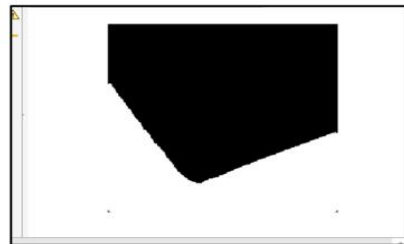
The bottom right corner of this photograph shows how the fabric is folded uniformly. This approach will reveal a textural difference.

Normal

Original Image



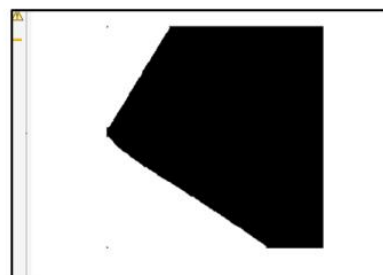
Final Result



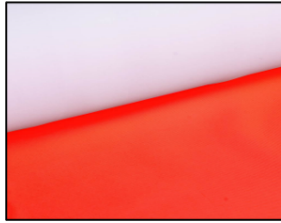
Original Image



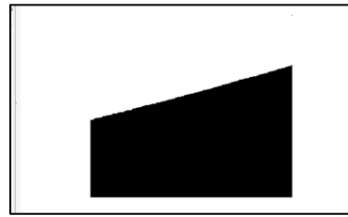
Final Result



Original Image



Final Result



These are the images with no defects and because of that the output will be like this.

Challenges

Lighting and shadows: It may be difficult to see fabric faults due to variations in lighting and shadows. Since faults can appear differently depending on the lighting, it is challenging to develop a consistent detection method.

Texture and color variations: Fabrics can have both texture and colour variations, which can make it difficult to identify defects in them. Because some flaws could be more obvious on specific types of fabrics or in specific areas of the fabric, a detection system that can be tailored to different fabrics is required.

Conclusion

Finding fabric faults with MATLAB can be an effective method, but it necessitates careful consideration of the particular challenges involved. MATLAB's image processing tools, like as filters, thresholding, and object removal methods, can be used to find flaws in fabric photos. However, it's important to consider the wide range of fabric textures, colours, and patterns as well as the numerous flaws that could develop in materials.

The likelihood of false positives and false negatives, as well as the scalability and cost-effectiveness of the detection system, must all be balanced. If these factors are carefully considered, the textile sector can profit from a MATLAB-based approach to fabric fault detection.